

Министерство науки и высшего образования Российской Федерации

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»

Институт информатики, математики и электроники

Факультет информатики

Кафедра технической кибернетики

Отчет по курсу

Дисциплина: «Технологии сетевого программирования»

Выполнили: Иванова Д. М.,

Мещеряков С.Е

Группа: 6301-010302D

Самара, 2025

СОДЕРЖАНИЕ

1 Концепция приложения	3
1.1 Назначение приложения	3
1.2 Состав приложения	3
1.3 Описание функциональных частей	3
1.3.1 Серверная часть	3
1.3.2 Клиентская часть	3
1.3.3 База данных	5
1.3.4 Нейросеть	5
1.4 Функционирование приложения	5
2 Работа приложения	6
2.1 Регистрация и авторизация	6
2.2 Генерация изображений	8
3 Ссылка на репозиторий GitHub	11

1 Концепция приложения

1.1 Назначение приложения

Приложение предназначено для генерации изображений по текстовому описанию.

1.2 Состав приложения

Приложение состоит из 4 частей:

- 1) Серверная часть приложения.
- 2) Клиентская часть приложения.
- 3) База данных.
- 4) Нейросеть.

1.3 Описание функциональных частей

1.3.1 Серверная часть

Серверная часть приложения написана на языке программирования Python с использованием фреймворка Django.

Также в ходе работы использовался Django REST Framework для написания методов API.

1.3.2 Клиентская часть

Клиентская часть реализована на React JS + css. В папке styles в css файлах хранятся стили для светлой и темной (пока нету) тем и для кастомных модулей – кнопок и форм ввода. Реализация формы ввода представлена на рисунке 1. Пример css файла представлен на рисунке 2. Страницы приложения хранятся в папке pages: страница регистрации, входа, главная страница, страница смены пароля. В папке router задаются пути и соответствующие им страницы. Пример приведен на рисунке 3.

```
import React from "react";
import style from '../../styles/MyInput.module.css';

const MyInput = React.forwardRef((props, ref) => {
  return (
    <input ref={ref} {...props} className={style.MyInput}/>
  );
});

export default MyInput;
```

Рисунок 1 – JSX файл с кодом для кастомной формы ввода

```
.MyInput {
  border-radius: 15px;
  width: 90%;
  height: 47px;
  margin-top: 5%;
  margin-top: 20px;
  border: none;
  outline: none;
}

input {
  font-family: 'Montserrat', 'semi-bold';
  font-size: 0.75rem;
  text-indent: 20px;
}

::-webkit-input-placeholder {
  /* WebKit/Blink browsers */
  font-family: 'Montserrat', 'semi-bold';
  font-size: 0.75rem;
}

::-moz-placeholder {
  /* Firefox 19+ */
  font-family: 'Montserrat', 'semi-bold';
  font-size: 0.75rem;
}

:-ms-input-placeholder {
  /* Internet Explorer 10-11 */
  font-family: 'Montserrat', 'semi-bold';
  font-size: 0.75rem;
}

:-moz-placeholder {
  /* Mozilla Firefox 4 to 18 */
  font-family: 'Montserrat', 'semi-bold';
  font-size: 0.75rem;
}

::placeholder {
  font-family: 'Montserrat', 'semi-bold';
  font-size: 0.75rem;
}
```

Рисунок 2 – CSS файл со стилями для формы ввода

```

//import ChangeCode from "../pages/ChangeCode.jsx";
import ChangePass from "../pages/ChangePass.jsx";
//import CodeLight from "../pages/Light/CodeLight.jsx";
//import Links from "../pages/Links.jsx";
import Login from "../pages/Login.jsx";
import Main from "../pages/Main.jsx";
//import OurTeam from "../pages/OurTeam.jsx";
import Registration from "../pages/Registration.jsx";

export const privateRoutes = [
]

export const publicRoutes = [
  {path: '/login', component: Login, exact: true},
  {path: '/registration', component: Registration, exact: true},
  //{path: '/code', component: CodeLight, exact: true},
  {path: '/main', component: Main, exact: true},
  //{path: '/change-code', component: ChangeCode, exact: true},
  {path: '/change-pass', component: ChangePass, exact: true},
  //{path: '/ourteam', component: OurTeam, exact: true},
  //{path: '/links', component: Links, exact: true}
]

```

Рисунок 3 – JS файл со стилями для формы ввода

1.3.3 База данных

В качестве СУБД была выбрана PostgreSQL. Для работы с базой использовались ORM модели, предлагаемые фреймворком Django.

1.3.4 Нейросеть

Нейросеть написана с помощью PyTorch. Для коммуникации с нейросетью используется единственный API метод, реализованный с помощью фреймворка FastAPI.

1.4 Функционирование приложения

Приложение собрано через docker-compose.yml файл. Образы функциональных частей в сумме весят 11,5 гигабайт.

2 Работа приложения

2.1 Регистрация и авторизация

Для получения доступа к основному функционалу приложения необходим вход в учётную запись. Если учётной записи не существует, то её можно зарегистрировать.

Авторизация выполнена с помощью стандартных инструментов фреймворка Django. В качестве модели пользователя используется своя модель пользователя, наследованная от `AbstractUser`. Для авторизации пользователя используются встроенные методы `TokenObtainPairView` и `TokenRefreshView`.

```
class User(AbstractUser):  
    id = models.AutoField(primary_key=True)  
    email = models.EmailField(unique=True)  
  
    def __str__(self):  
        return self.userName
```

Рисунок 4 – Модель пользователя

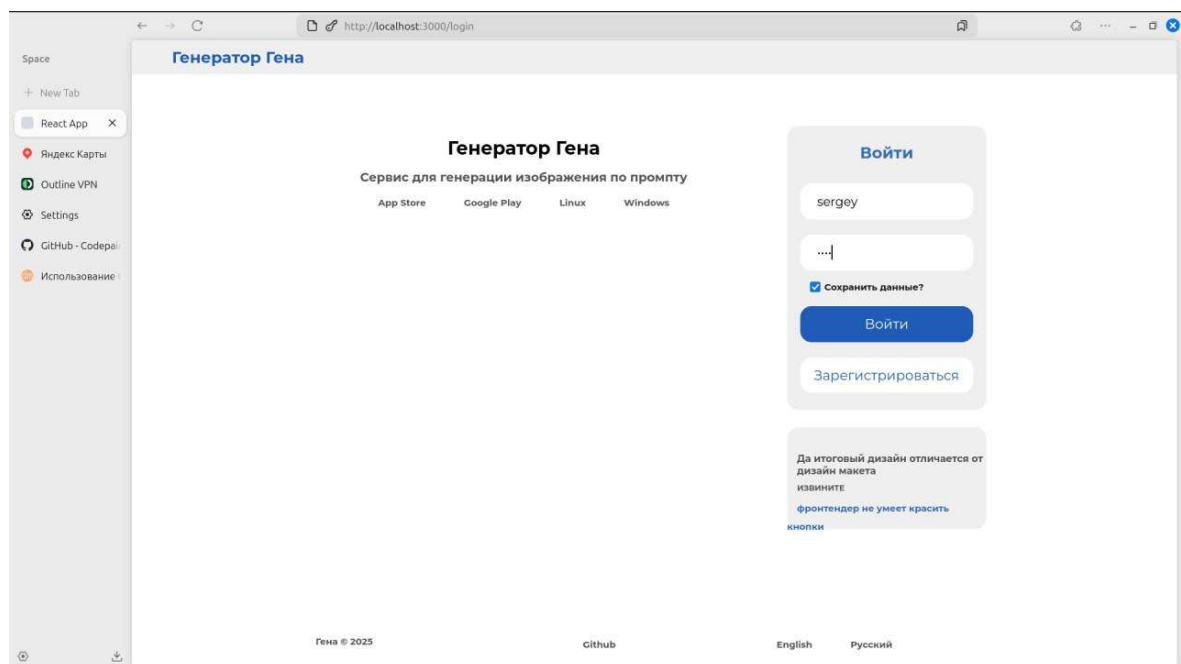


Рисунок 5 – Страница входа

Регистрация пользователя происходит через сериализатор, получающий значения полей и регистрирующий пользователя в базе данных.

```
class RegisterUserView(APIView):
    permission_classes = [AllowAny]
    def post(self, request):
        serializer = UserSerializer(data=request.data)
        try:
            if serializer.is_valid():
                serializer.save()
                return Response(serializer.data, status=status.HTTP_201_CREATED)
            else:
                return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
        except Exception as e:
            logger.error(f"Ошибка регистрации пользователя: {str(e)}")
            return Response({"error": "Internal Server Error"}, status=status.HTTP_500_INTERNAL_SERVER_ERROR)
```

Рисунок 6 – Регистрация пользователя

```
class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['id', 'username', 'email', 'password']
        extra_kwargs = {'password': {'write_only': True}}

    def create(self, validated_data):
        user = User.objects.create_user(**validated_data)
        return user

    def update(self, instance, validated_data):
        if 'password' in validated_data:
            instance.set_password(validated_data.pop('password'))
        return super().update(instance, validated_data)
```

Рисунок 7 – Сериализатор для пользователя

Для регистрации пользователю необходимо указать логин, пароль и адрес электронной почты. Проверка пароля осуществляется неявно внутренними средствами фреймворка.

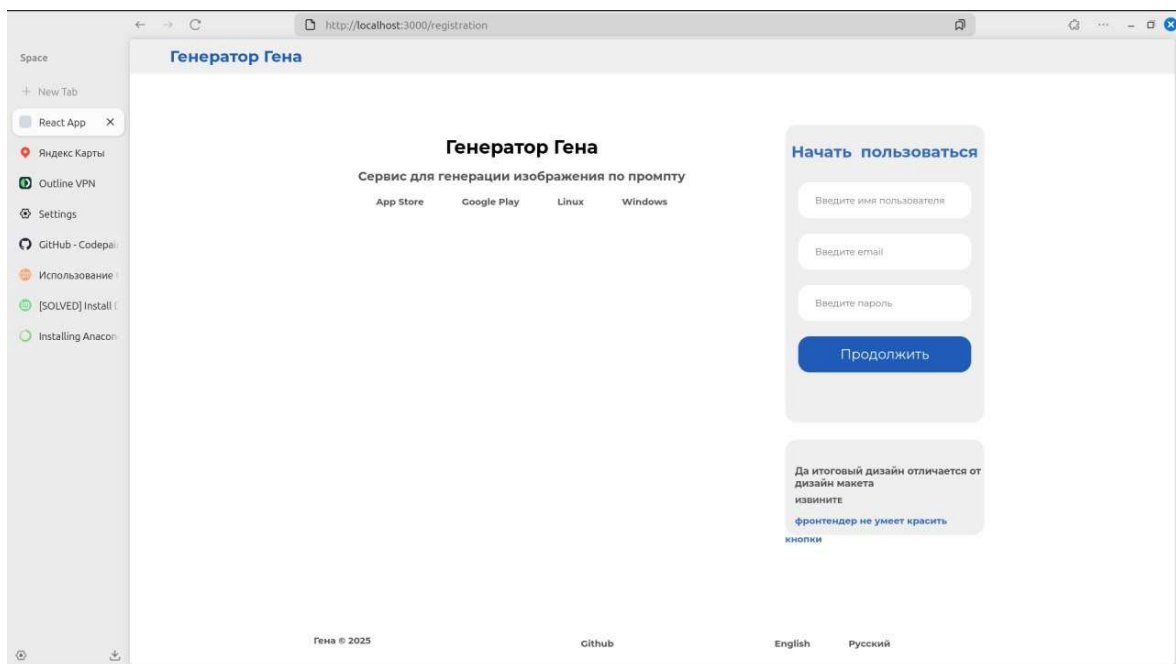


Рисунок 8 – Страница регистрации

2.2 Генерация изображений

На главной странице сайта доступно поле ввода для генерации изображений. Текст должен быть на английском языке.

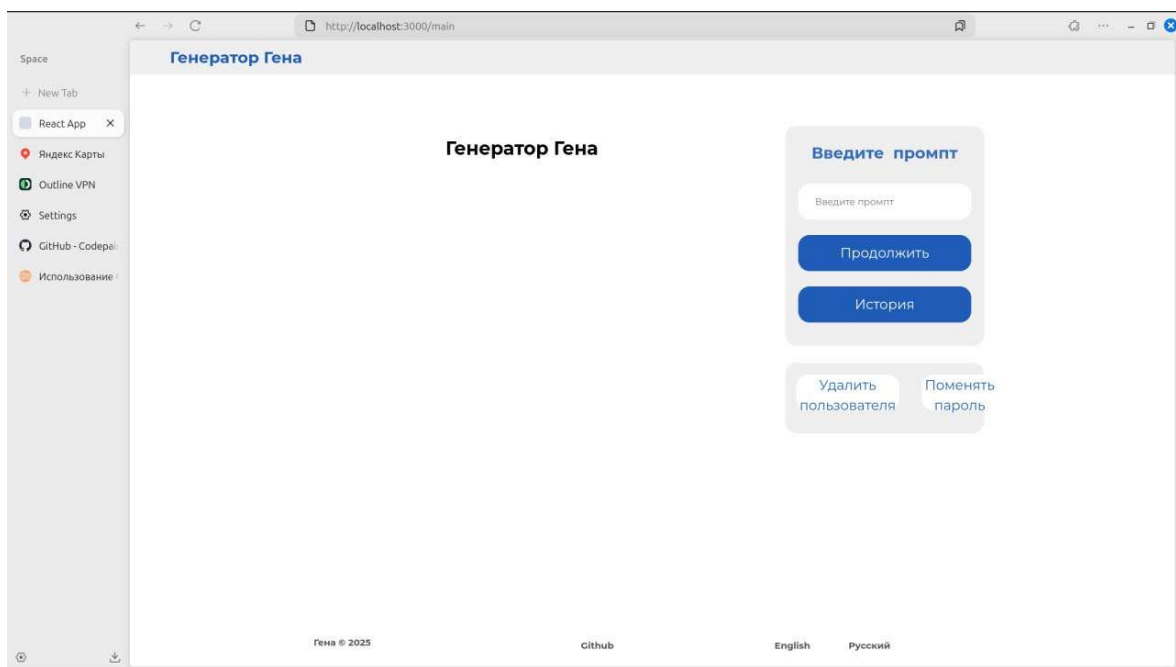


Рисунок 9 – Главная страница

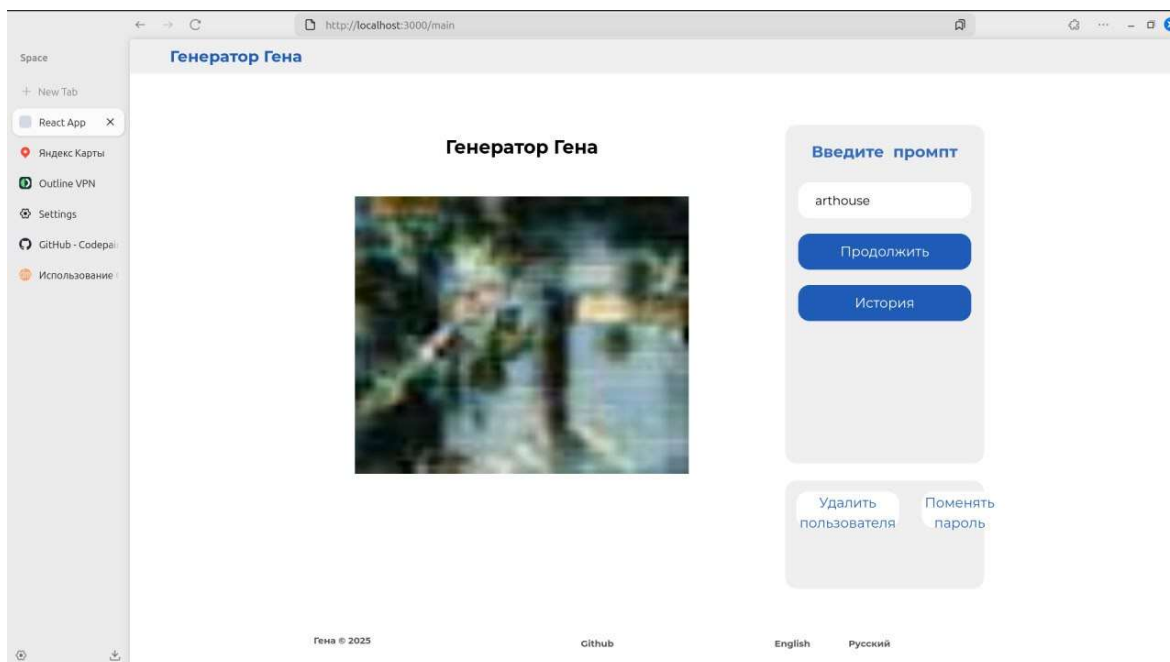


Рисунок 10 – Сгенерированное изображение

В процессе выполнения задания тестировалось несколько архитектур модели. Из-за ограничений ресурсов на данный момент используется самая простая, генерирующая изображения с разрешением 64x64. Основной проблемой при обучении моделей был mode collapse – генератор либо находил несколько изображений, с которыми можно обманывать дискриминатор, либо дискриминатор оказывался слишком сильным и генератор сваливался в генерацию случайного шума даже после 60 эпох обучения. Код сделан таким образом, что модели можно легко заменять корректировкой одной строчки с загрузкой весов модели и использования соответствующего кода для архитектуры модели. Модель вынесена как отдельное FastAPI приложение.

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from gena import Gena
import traceback

app = FastAPI()
model = Gena()

class GenerateRequest(BaseModel):
    prompt: str

@app.post("/generate/")
def generate_image(data: GenerateRequest):
    try:
        result = model.generate_from_text(data.prompt)
        return {"images": result}
        #print(f"Received prompt: {data.prompt}")
        # временный фейковый ответ
        #return {"images": ["base64_stub_image_data"]}
    except Exception as e:
        print("Error while generating image:")
        traceback.print_exc()
        raise HTTPException(status_code=500, detail=str(e))

```

Рисунок 11 – FastAPI приложение

```

class Generator(nn.Module):
    def __init__(self, noise_dim=100, text_dim=512, out_channels=3):
        super().__init__()
        self.projection = nn.Linear(text_dim, 256)
        self.fc = nn.Linear(noise_dim + 256, 512 * 4 * 4)

        self.net = nn.Sequential(
            nn.BatchNorm2d(512),
            nn.ConvTranspose2d(512, 256, 4, 2, 1), # 8x8
            nn.BatchNorm2d(256),
            nn.ReLU(True),
            nn.ConvTranspose2d(256, 128, 4, 2, 1), # 16x16
            nn.BatchNorm2d(128),
            nn.ReLU(True),
            nn.ConvTranspose2d(128, 64, 4, 2, 1), # 32x32
            nn.BatchNorm2d(64),
            nn.ReLU(True),
            nn.ConvTranspose2d(64, out_channels, 4, 2, 1), # 64x64
            nn.Tanh()
        )

    def forward(self, noise, text_emb):
        projected_text = self.projection(text_emb)
        x = torch.cat([noise, projected_text], dim=1)
        x = self.fc(x).view(-1, 512, 4, 4)
        return self.net(x)

```

Рисунок 12 – Архитектура генератора

3 Ссылка на репозиторий GitHub

<https://github.com/Codepairs/generatorgena>